github.com/mg-i-roc-maz/corsi · microservizi/lezione-5

LEZIONE 5 DI 8 · 3 ORE

Caching: i fondamenti

Rispondere in 3 millisecondi invece che in 100, senza mostrare al cliente un prezzo sbagliato.

Le tre ore di oggi

Teoria · 1h15 | **Laboratorio · 1h35**

Teoria

Hit e miss, TTL, cache-aside e gli altri pattern, invalidazione, dati vecchi, Redis, eviction, cosa mettere in cache.

Laboratorio

Misuriamo catalog senza cache, scriviamo il cache-aside, aggiungiamo Redis al Compose, invalidiamo, guardiamo dentro Redis.

Ultimi 10'

Una domanda per chiudere: «cosa si rompe se…?»

DA DOVE RIPARTIAMO

Lezione 4: il fallback con «l'ultimo prezzo noto» era già una cache.

UNA COPIA

Un dato letto prima, tenuto da parte per non chiederlo di nuovo.

QUANTO VECCHIA?

Il prezzo può essere cambiato nel frattempo. Fino a quando ci fidiamo?

CHI LA AGGIORNA?

Quando il dato cambia, qualcuno deve buttare la copia.

PERCHÉ UNA CACHE

La lettura più veloce è quella che non fai.

Una cache tiene una **copia** di un dato costoso da ottenere in un posto **veloce** da leggere. Si paga in memoria e in possibili dati vecchi.

Ordini di grandezza

Operazione	Tempo tipico	Se fosse 1 secondo…
Leggere dalla RAM del processo	`~100 ns`	1 secondo
GET a Redis nella stessa rete	`~0,3 ms`	50 minuti
Query semplice su PostgreSQL, con indice	`~1 ms`	3 ore
Query pesante, o database sotto carico	`10–1000 ms`	giorni
Chiamata a un'API esterna	`50–300 ms`	settimane

Nel laboratorio catalog simula una query da **100 ms** (DB_DELAY_MS): con Redis la stessa lettura scende a **pochi ms**.

Dove può stare una cache



Più la cache è **vicina al cliente**, più fa risparmiare, e più è difficile invalidarla.

Livello	Quando
Applicazione + Redis	Oggi: catalog decide cosa salvare e quando buttarlo
HTTP: browser, gateway, CDN	Lezione 6: ETag, Cache-Control, cache di nginx

Hit e miss

HIT

Il dato è in cache: si risponde subito, il database non viene toccato.

`X-Cache: HIT · 3 ms`

MISS

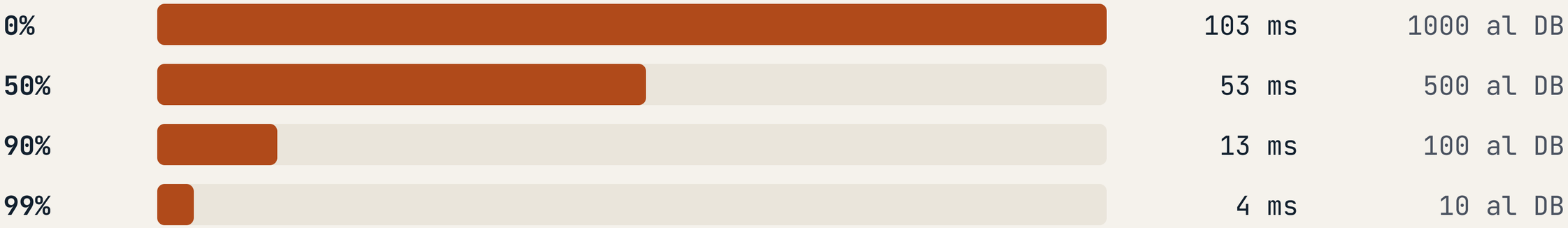
Il dato non c'è, o è scaduto: si va al database e si salva la copia per la prossima volta.

`X-Cache: MISS · 103 ms`

```
hit ratio = hit / (hit + miss)
```

Quanto conta l'hit ratio

HIT 3 ms, MISS 103 ms. Tempo medio di 1000 letture:



Il guadagno vero spesso non è la velocità: è il **carico tolto al database**. Al 90% riceve un decimo delle query.

TTL: quanto vive una copia

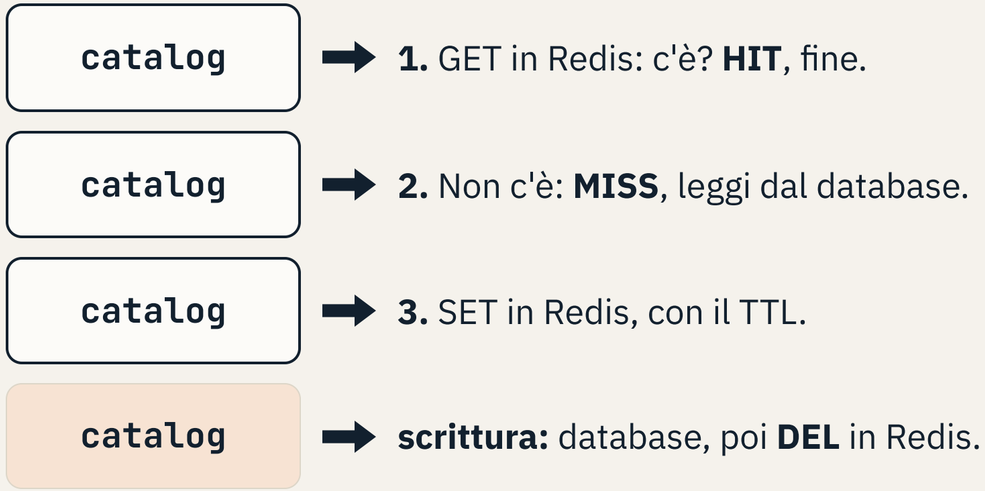
```
SET product:p-1 "{...}" EX 60
TTL product:p-1 # 57
# dopo 60 secondi Redis la cancella da solo
GET product:p-1 # (nil)
```

Time To Live: dopo il TTL la copia sparisce e la lettura successiva è un MISS.

TTL	Effetto
corto	dati più freschi, più MISS, più carico sul DB
lungo	più HIT, ma un dato cambiato resta vecchio più a lungo

Il TTL è la **rete di sicurezza**: anche se tutto il resto fallisce, un dato vecchio prima o poi sparisce. Mettilo sempre.

Cache-aside: la cache «a lato»



È il codice di catalog a gestire tutto: Redis non sa niente del database.

Il pattern più usato, ed è quello di oggi.

Cache-aside nel codice

```
const key = `product:${id}`;
const cached = await cache.get(key);
if (cached !== null) return { product: JSON.parse(cached), cache: "HIT" };

const product = await store.get(id); // il database
if (product) await cache.set(key, JSON.stringify(product), ttlSeconds);
return { product, cache: "MISS" };

// dopo PUT, DELETE, prenotazione:
await cache.del(key);
```

Redis salva **stringhe**: l'oggetto si trasforma in JSON per salvarlo e si rilegge con JSON.parse.

Quattro modi di usare una cache

Pattern	Lettura	Scrittura
Cache-aside	L'app guarda in cache, se manca legge il DB e salva	L'app scrive nel DB e cancella la copia
Read-through	L'app chiede solo alla cache; è la cache a leggere il DB	come cache-aside
Write-through	come cache-aside	Si scrive nel DB **e** in cache, subito
Write-behind	come cache-aside	Si scrive in cache; il DB viene aggiornato **dopo**

⚠ Write-behind è il più veloce e il più rischioso: se la cache si spegne prima di aver scritto nel database, le modifiche sono **perse**.

Invalidare o aggiornare?

Cancellare (oggi)

```
UPDATE products ...
DEL product:p-1
```

Semplice e sicuro: la prossima lettura prende il dato vero. Costa un MISS.

Aggiornare (write-through)

```
UPDATE products ...
SET product:p-1 "{nuovo}" EX 60
```

Nessun MISS. Ma se due scritture si incrociano, in cache può restare quella **sbagliata**.

In caso di dubbio: **cancella**. Un MISS in più costa poco; un dato sbagliato in cache può costare molto.

INVALIDAZIONE

«Ci sono solo due cose difficili in informatica: invalidare la cache e dare i nomi alle cose.»

Phil Karlton

Come si butta una copia

Strategia	Come	Limite
Solo TTL	Si aspetta che scada	Dato vecchio fino al TTL
Cancella alla modifica	Chi scrive nel DB fa DEL della chiave	Deve conoscere tutte le chiavi coinvolte
Aggiorna alla modifica	Chi scrive nel DB fa SET del valore nuovo	Scritture incrociate
Eventi	Chi scrive pubblica «p-1 è cambiato»; chi ha copie le butta	Serve un broker: lezione 7

Oggi: **cancella alla modifica + TTL**. Il TTL copre tutto ciò che la cancellazione non vede.

Una modifica, quante chiavi?

```
product:p-1 # il prodotto
products:all # la lista (extra 3)
search:mouse # una ricerca
category:5:page:1 # una pagina di categoria
```

Cambia il prezzo di p-1: **quali** di queste chiavi contengono il prezzo vecchio?

Regola pratica

Metti in cache cose che sai **invalidare**. Se non sai dire quali chiavi cambiano a una modifica, usa solo un TTL breve.

Quanto vecchio può essere un dato?

Dato	Vecchio di 60 s è…	Perché
Descrizione prodotto	**accettabile**	Nessuno se ne accorge
Prezzo in vetrina	**accettabile, con cautela**	Il prezzo vero lo legge orders all'ordine
Scorte in vetrina	**accettabile**	La prenotazione va comunque al DB
Scorte per prenotare	**mai**	Venderemmo ciò che non c'è
Saldo di un conto	**mai**	Soldi veri

La domanda non è tecnica: è «**cosa succede se il cliente vede il dato vecchio?**»

Anche il cache-aside sbaglia, a volte

t1	`A: GET cache → MISS`	
t2	`A: legge dal DB → 7990`	
t3		`B: UPDATE prezzo → 8990`
t4		`B: DEL product:p-1`
t5	`A: SET product:p-1 = 7990`	

A era lento e scrive in cache il valore **vecchio**, dopo la cancellazione di B. Resta lì fino al TTL.

Redis

Un database **chiave → valore** che tiene tutto in **memoria**.

- risponde in frazioni di millisecondo
- un solo thread per i comandi: ogni comando è **atomico**
- ogni chiave può avere una **scadenza**
- condiviso da tutte le copie di catalog

Tipo	Esempio d'uso
string	un prodotto in JSON, un contatore
hash	un oggetto, campo per campo
list	una coda di lavori
set	i tag di un prodotto
sorted set	una classifica, i più venduti

Oggi usiamo solo le **string**.

I comandi di oggi

Comando	Cosa fa
`SET product:p-1 "{...}" EX 60`	salva, con scadenza in secondi
`GET product:p-1`	legge; (nil) se non c'è
`TTL product:p-1`	secondi rimasti; -2 = non esiste, -1 = nessuna scadenza
`DEL product:p-1`	cancella
`KEYS product:*`	elenca le chiavi (solo per provare!)
`INFO stats`	statistiche: keyspace_hits, keyspace_misses
`MONITOR`	mostra in diretta ogni comando ricevuto

```
docker compose exec redis redis-cli
```

Quando la memoria finisce

```
redis-server
  --save "" --appendonly no
  --maxmemory 64mb
  --maxmemory-policy allkeys-lru
```

Oltre 64 MB Redis butta le chiavi usate **meno di recente** (LRU) per fare spazio.

Policy	Butta…
noeviction	niente: errore alle scritture
allkeys-lru	le meno usate di recente
allkeys-lfu	le usate meno spesso
volatile-ttl	quelle che scadono prima

Per una cache: **allkeys-lru**. Una chiave buttata è solo un MISS.

La cache è un'ottimizzazione, non una dipendenza

Redis giù

- la lettura in cache fallisce
- catalog legge dal database: **BYPASS**
- più lento, ma funziona
- readiness: ok, con **cache: down**

Redis lento

- ogni comando ha un **timeout di 200 ms**
- disableOfflineQueue: niente comandi in coda
- oltre: errore, poi BYPASS

⚠ Lezione 4, di nuovo: lento è peggio di spento. Una cache che aspetta 5 secondi è peggio di nessuna cache.

Cache stampede: tutti insieme al database



Una chiave molto letta scade: tutte le richieste di quell'istante fanno MISS e vanno al database **insieme**.

Rimedio	Idea
Lock o «single flight»	Solo una richiesta legge il DB; le altre aspettano il suo risultato
Jitter sul TTL	Chiavi salvate insieme non scadono insieme (extra 2)
Rinnovo anticipato	Si ricarica la copia poco prima che scada

Cache locale o distribuita?

	In memoria del processo	Redis
Velocità	la massima: niente rete	~0,3 ms in più
Con 3 copie di catalog	3 cache diverse, 3 MISS	una sola cache condivisa
Invalidazione	va fatta in ogni copia	un DEL basta
Riavvio di catalog	cache persa	cache intatta
Memoria	rubata al servizio	un servizio a parte

Nel laboratorio ci sono tutte e due: **memoria** nei test, **Redis** nel Compose. Stessa interfaccia.

Cosa mettere in cache

Buoni candidati

- letto spesso, cambiato di rado
- costoso da calcolare o da leggere
- uguale per tutti gli utenti
- accettabile se vecchio di un po'

Catalogo, categorie, configurazioni, pagine pubbliche.

Cattivi candidati

- cambia a ogni richiesta
- diverso per ogni utente, letto una volta
- deve essere sempre esatto
- già veloce da leggere

Saldo, scorte per prenotare, stato di un pagamento.

Prima **misura**, poi metti in cache. Una cache che non serve è solo un posto in più dove nascondere bug.

Verifichiamo: hit, miss, pattern

1. HIT 2 ms, MISS 102 ms, hit ratio 80%: qual è il tempo medio?
2. Nel cache-aside, chi legge il database: l'app o la cache?
3. Perché il TTL va messo anche se invalidiamo a ogni modifica?
4. Perché write-behind non va bene per gli ordini?

Verifichiamo: invalidazione e Redis

1. Dopo una PUT, perché cancellare la copia invece di aggiornarla?
2. Redis è giù: cosa risponde catalog, e con che tempi?
3. Perché il Redis del laboratorio non ha un volume?
4. Cosa succede con maxmemory raggiunto e allkeys-lru?

Glossario

Termine	Significato
Hit, miss	Il dato è in cache, oppure no e va letto dalla fonte.
Hit ratio	La percentuale di letture servite dalla cache.
TTL	Time To Live: dopo quanto una copia scade.
Cache-aside	L'app legge la cache, se manca legge il DB e salva la copia.
Write-through, write-behind	Scrivere in cache insieme al DB, oppure prima del DB.
Invalidazione	Buttare una copia perché il dato è cambiato.
Eviction, LRU	Buttare copie quando la memoria finisce; le meno usate di recente.
Cache stampede	Molte richieste fanno MISS insieme e vanno tutte al DB.

Laboratorio

10'

1 · Senza cache

Quanto costa una lettura?

25'

2 · Cache-aside

TODO 1, con i test.

15'

3 · Redis

TODO 5 e 6: nel Compose.

20'

4 · Invalidazione

TODO 2, 3 e 4.

15'

5 · Dentro Redis

redis-cli, dati vecchi, Redis spento.

```
cd microservizi/lezione-5/start → docker compose up --build -d
```

Il repo della lezione

`github.com/mg-i-roc-maz/corsi`

```
cd corsi
git pull
cd microservizi/lezione-4/start
docker compose down
cd ../../lezione-5/start
docker pull redis:7.4
docker compose up --build -d
```

microservizi/lezione-5/

Cartella	Contenuto
`slides.pdf`	Queste slide, in PDF
`start/`	**Si lavora qui**
`solution/`	La soluzione, per riallinearsi
`extra/`	Esercizi extra, test e soluzioni

I passi sono anche in **SETUP.md**, da fare prima della lezione.

Cosa c'è in start

```
gateway/nginx.conf              nuovo: /stats/cache
services/catalog/src/
  cache.ts                      Redis o memoria, già pronto
  product-cache.ts              TODO 1
  routes.ts                     TODO 2, 3, 4
  store/slow.ts                 DB lento: DB_DELAY_MS
compose.yaml                    TODO 5, 6
scripts/misura.sh               tempi e X-Cache
```

È la soluzione della lezione 4, più la cache.

Nuova dipendenza in catalog: **redis**, il client ufficiale.

```
cd services/catalog
npm install
```

1 · Quanto costa una lettura?

```
./scripts/misura.sh
# GET http://localhost:8080/api/v1/products/p-1
# 1 BYPASS 104 ms
# 2 BYPASS 103 ms
# ...
# {"hits":0,"misses":0,"errors":0,...}
```

catalog simula una query lenta: **100 ms** per ogni lettura dal database (DB_DELAY_MS).

BYPASS: la cache ancora non c'è. Annota i tempi.

Windows: lancia lo script da Git Bash, o invia più volte la richiesta da http/shop.http.

2.1 · I test di oggi

```
cd services/catalog
npm install
npm test
```

Nessun Docker: i test usano la cache **in memoria**, con la stessa interfaccia di Redis.

ALL'INIZIO

```
# pass 21 · # fail 5
```

DOPO IL TODO 1

```
# pass 23 · # fail 3
```

DOPO LA PARTE 4

```
# pass 26 · # fail 0
```

2.2 · TODO 1: il cache-aside

In **getProduct**, dentro src/product-cache.ts:

1. **cache.get(key)**, in un try/catch: se fallisce, stats.errors++ e si va al database
2. valore trovato: stats.hits++, restituisci **JSON.parse** e **"HIT"**
3. altrimenti **store.get(id)**; se la cache era giù: **"BYPASS"**
4. se no: stats.misses++, **cache.set** con il TTL (solo se il prodotto esiste) e **"MISS"**

Verifica

```
npm test
```

Passano «MISS poi HIT» e «un HIT non tocca il database». Restano rossi i tre sull'invalidazione.

2.3 · Chi usa getProduct

```
// routes.ts: GET /products/:id
const { product, cache } =
  await productCache.getProduct(store, id);
reply.header("X-Cache", cache);
if (!product) return reply.code(404)...
return product;
```

La rotta è già pronta: chiama getProduct e scrive nell'header **X-Cache** come è andata.

GET /cache/stats restituisce hits, misses, errors e hit ratio. Dal gateway: **/stats/cache**.

3.1 · TODO 5 e 6: Redis nel Compose

```
redis:
  image: redis:7.4
  command: ["redis-server", "--save", "",
    "--appendonly", "no", "--maxmemory", "64mb",
    "--maxmemory-policy", "allkeys-lru"]
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
  restart: unless-stopped

catalog:
  environment:
    REDIS_URL: redis://redis:6379
    CACHE_TTL_SECONDS: ${CACHE_TTL_SECONDS:-60}
  depends_on:
    redis: { condition: service_started }
```

L'host è il **nome del servizio**: redis. La porta di Redis è **6379**.

Perché service_started e non service_healthy?

E perché niente volume?

3.2 · Di nuovo la misura

```
docker compose up --build -d
docker compose logs catalog | grep cache
./scripts/misura.sh
# 1 MISS 128 ms
# 2 HIT 4 ms
# 3 HIT 3 ms
# ...
# {"hits":9,"misses":1,"hitRatio":0.9,...}
```

Il primo è un **MISS**, poi tutti **HIT**.

Quante volte più veloce? Rilancia lo script dopo 60 secondi: cosa cambia?

Prova anche p-2: il suo primo accesso è un MISS.

4.1 · Cambia un prezzo

```
# da http/shop.http
GET /api/v1/products/p-1 # 8990, HIT
PUT /api/v1/products/p-1 # priceCents 7990
GET /api/v1/products/p-1 # ???
```

La PUT risponde con il prezzo nuovo. E la GET subito dopo?

Domanda

Per quanto tempo il cliente vedrà il prezzo vecchio?
Da cosa dipende?

4.2 · TODO 2, 3 e 4: invalidare

```
// PUT, DELETE e prenotazione, dopo il successo:
await productCache.invalidate(request.params.id);
```

TODO	Rotta	Perché
2	PUT /products/:id	il prodotto è cambiato
3	DELETE /products/:id	non esiste più
4	POST .../reservations	le scorte sono cambiate

Una riga, tre volte. La parte difficile è **non dimenticarne nessuna**.

Poi: **npm test** verde, ricostruisci, ripeti la prova della slide precedente.

5.1 · Dentro Redis

```
docker compose exec redis redis-cli
> KEYS product:*
> GET product:p-1
> TTL product:p-1
> INFO stats # keyspace_hits, _misses
> MONITOR # Ctrl+C per uscire
```

Con **MONITOR** acceso, lancia misura.sh da un altro terminale: cosa vedi per un MISS? E per un HIT?

Con **TTL** guarda il contatore scendere. Cosa risponde dopo la scadenza?

5.2 · Due esperimenti

Il dato vecchio

Leggi p-2, poi cambia il prezzo **scavalcando catalog**:

```
docker compose exec catalog-db \
  psql -U shop -d catalog -c \
  "UPDATE products SET price_cents = 1
   WHERE id = 'p-2';"
```

Quando vedi il prezzo nuovo? Perché?

Redis spento

```
docker compose stop redis
./scripts/misura.sh
docker compose start redis
```

catalog risponde? Con che tempi e che X-Cache? E /health/catalog?

Checkpoint

Verifica	Esito atteso
`npm test in services/catalog`	pass 26, fail 0, 4 saltati
`./scripts/misura.sh`	un MISS da ~100 ms, poi HIT da pochi ms
`PUT e poi GET dello stesso prodotto`	il prezzo nuovo, X-Cache: MISS
`TTL product:p-1 in redis-cli`	un numero tra 0 e 60
`docker compose stop redis`	catalog risponde, X-Cache: BYPASS
`GET /stats/cache`	hits, misses, hit ratio

Consegna: l'output di misura.sh prima e dopo la cache, e la risposta all'esperimento del dato vecchio.

Se qualcosa non va

Sintomo	Causa e rimedio
`X-Cache sempre BYPASS`	TODO 1 non fatto, o catalog non ricostruito: --build
`Sempre MISS, mai HIT`	La set non avviene: manca await o JSON.stringify
`Nel log c'è «cache memoria»`	REDIS_URL non impostato nel servizio catalog
`Redis non disponibile all'avvio`	Normale per un attimo; se resta, docker compose ps
`GET risponde 500 dopo un HIT`	JSON.parse su un valore salvato male: DEL della chiave
`Il prezzo cambiato non si vede`	Manca l'invalidazione: TODO 2, o aspetta il TTL
`X-Cache non compare`	Stai leggendo la lista: c'è solo su /products/:id

ULTIMI 10 MINUTI

Cosa si rompe se...?

...catalog gira in tre copie e la cache è in memoria del processo, non in Redis?

...orders legge il prezzo da una sua cache dei prodotti, con TTL di un'ora?

...il prodotto più visto scade alle 21:00, durante uno spot in TV?

PROSSIMA LEZIONE

Lezione 6 · Caching nei microservizi

Cache HTTP con ETag e Cache-Control, cache nel gateway nginx, invalidazione tra servizi. E lo stampede: lo provochiamo con k6 e lo mitighiamo.

Per la volta prossima: npm test verde in catalog e misura.sh con i HIT.

PER CHI FINISCE PRIMA, O PER CASA

Esercizi extra

#	Esercizio	Livello	Dove
1	Ricordarsi di ciò che non esiste	base	catalog
2	Scadenze sparse: TTL con jitter	base	catalog
3	La lista in cache	medio	catalog
4	Write-through	medio	catalog
5	Cosa mettere in cache nel negozio	carta e penna	—

Testi in lezione-5/extra/README.md. Per 1–4 ci sono test pronti; le soluzioni sono in lezione-5/extra/soluzione.

Ricordarsi di ciò che non esiste

Venti richieste di **p-999**: venti query al database. Un 404 non finisce in cache.

- opzione **negativeTtlSeconds**, default 0
- salva un valore speciale: **"__non_esiste__"**
- trovato in cache: 404 con **X-Cache: HIT**
- variabile: **CACHE_NEGATIVE_TTL_SECONDS**

Domanda

Perché il TTL negativo deve essere molto più corto di quello normale?

Scadenze sparse: TTL con jitter

Un job legge tutti i prodotti alle 8:00:00. Con TTL 60 scadono **tutti insieme** alle 8:01:00.

- opzione **ttlJitterPercent**: con 10, TTL tra 54 e 66 s
- per i test: **random**, default Math.random
- variabile: **CACHE_TTL_JITTER_PERCENT**

Verifica

Leggi p-1, p-2, p-3 e confronta i **TTL** in redis-cli. È lo stesso jitter della lezione 4.

La lista in cache

GET /products senza ricerca è la pagina più vista, e va sempre al database.

- una sola chiave: **products:all**, con X-Cache
- le ricerche (**?q=**) non passano dalla cache
- quando va buttata la lista?

Domanda

Mille prodotti, una modifica al secondo: quanto serve questa cache?

Write-through

Dopo una PUT, invece di cancellare la copia, **scrivi il valore nuovo** in cache.

- metodo **write(product)** in product-cache.ts
- usalo nella PUT al posto di invalidate
- se la scrittura in cache fallisce?

Il test «PUT invalida la cache» accetta sia MISS sia HIT.

Domanda

Perché per le prenotazioni conviene comunque invalidare?

Cosa mettere in cache nel negozio

Per ogni dato: in cache sì o no, TTL, come si invalida, cosa succede se è vecchio.

Dato	Cache?
Nome e descrizione	
Prezzo	
Scorte	
Categorie	
Risultati di ricerca	
Stato di un ordine	
Carrello	

Spunto

Per ogni riga chiediti: **chi perde** se il dato è vecchio di un minuto?

Per approfondire

Redis Docs · *Data types* e *Commands*	redis.io/docs: string, hash, list, set; SET, GET, EXPIRE, SCAN
Redis Docs · *Key eviction*	maxmemory e le policy LRU, LFU, TTL
AWS Whitepaper · *Database Caching Strategies Using Redis*	cache-aside, write-through, TTL e stampede
M. Kleppmann · *Designing Data-Intensive Applications*	O'Reilly, 2017: repliche, consistenza, dati derivati
node-redis · documentazione	github.com/redis/node-redis: il client usato oggi
Valkey	valkey.io: il fork open source di Redis, compatibile